

# npuloop: 가상 NPU 비용 모델과 비트 정확 정수 엔진으로 달는 모델 압축 루프

기술 보고서 · 2026-09-14 · 저장소 [github.com/sokldjs554/npuloop](https://github.com/sokldjs554/npuloop)

## 초록

NPU 회사의 모델 팀은 고객이 보낸 체크포인트에 대해 "우리 칩에서 돌기는 하는가, 얼마나 걸리는가, 무엇을 바꿔야 하는가"를 답해야 한다. 이 보고서는 그 세 질문을 숫자로 답하는 오픈소스 툴킷 npuloop의 설계와 실험을 정리한다. torch.fx 그래프 IR 위에 (1) SCALE-Sim으로 검증한 weight-stationary systolic-array 비용 모델, (2) NPU 준비도 lint, (3) 비트 단위로 일치하는 두 정수 엔진(NumPy·C++)과 파이썬 없는 독립 실행기, (4) 비용 모델을 루프 안에서 호출하는 그래프 기반 구조적 프루닝을 엮었다. CIFAR-10 5개 모델(ResNet-20 ReLU/SiLU, MobileNetV2-0.5, Inception-32, ViT-128/6)과 Imagenette 128×128에서 fake-quant와 정수 실행의 불일치를 노드별로 국소(teacher-forced)/전파로 분리해 재고, 정수 구현 세부(반올림·곱셈기 비트·누산기·바이어스 폭)의 정확도 비용을 ablation했으며, TensorFlow Lite reference 커널과 1,000장 × 모든 텐서에서 비트 일치함을 보였다. 주요 결과: fake-quant는 정확도 예측기로 충분하지만(주 스킴에서  $|\Delta| \leq 0.20\%$ ) 텐서 단위로는 출력 코드의 17~84%가 다르고, transformer에서는 LayerNorm이 국소 불일치의 대부분을 차지하며, TFLite reference 커널은 conv/pool과 fully-connected가 서로 다른 반올림을 쓴다.

## 1. 문제

INT8 NPU에 모델을 올리는 작업은 세 층위의 불일치를 다룬다. (a) 학습 프레임워크의 fake-quant와 실제 정수 실행의 차이, (b) FLOPs와 사이클의 차이(배열 정렬·채움/비움·메모리), (c) 같은 "INT8"이라도 런타임마다 다른 requantization 구현. 공개 도구는 이 셋을 따로 다룬다: MQBench·PPQ는 (a)를 백엔드 정확도 격차로, SCALE-Sim·Timeloop는 (b)를 오프라인 시뮬레이션으로, TFLite·gemmlowp는 (c)를 코드로만. npuloop의 기여는 셋을 한 그래프 IR 위에 놓고, 결정(프루닝·활성함수 교체·PTQ/QAT)이 실제 정수 결과와 사이클로 되돌아오게 한 것이다.

## 2. 방법

**IR.** torch.fx 로 추적한 그래프에서 BN을 접고 shape를 전파한 StaticGraph.conv/linear(토큰 단위 포함)/add/mul/concat/matmul/softmax/ layernorm/transpose/reshape/pool/활성함수를 지원하고 나머지는 즉시 실패시킨다.

**비용 모델.** weight-stationary systolic array에서 GEMM 한 타일의 사이클을  $M + 2R + C - 2$  (M 출력 픽셀, R×C 배열)로 두고 [K/R]·[N/C] 타일을 합산, 멀티코어 M/N 분할, depthwise 엔진, LUT/융합/호스트 폴백 활성 함수, DRAM roofline을 더한다. SCALE-Sim v3(사이클 정확, WS)와 4개 모델 × 3개 배열 크기에서 합계 0.1% 이내로 일치한다(표 6). 에너지는 MAC·SRAM·DRAM·벡터·호스트 항목의 pJ 상수(Horowitz 2014 자릿수)로 같은 양에서 추정하며 simulated 라벨을 단다.

**정수 엔진.** 캘리브레이션된 fake-quant 그래프를 int8 가중치·int32 바이어스·Q31 곱셈기+시프트의 IntGraph로 export하고, gemmlowp/TFLite의 MultiplyByQuantizedMultiplier 의미론으로 실행한다. NumPy 구현과 C++ 커널(ctypes)은 모든 중간 텐서에서 비트 일치를 테스트로 강제하고, .npuloop 파일(JSON 헤더 + raw 정수 배열)을 파이썬 없이 실행하는 C++ 러너가 같은 답을 낸다. RequantConfig로 반올림 모드·곱셈기 비트·누산기/바이어스 폭을 바꿔 "짜구려 구현"을 흉내 낼 수 있고, 노드별 반올림 오버라이드도 가능하다.

**검증.** 같은 이미지에서 fake-quant 코드와 정수 코드를 노드별로 비교하되, 두 가지로 켜다: **전파 불일치**(끝까지 정수로 실행)와 **국소 불일치**(각 노드에 fake-quant의 입력 코드를 teacher-forcing으로 넣고 그 노드의 출력만 비교). 국소 값이 어느 연산이 차이를 *만들어내는지*를, 전파 값이 그 차이가 어디까지 *번지는지*를 보여 준다.

**프루닝.** 그래프에서 conv/linear 출력 채널을 활성함수·depthwise conv를 지나 소비자까지 따라가 그룹을 찾는다(residual add·pool·matmul·LayerNorm에 닿으면 제외, concat을 지나면 소비자의 입력 슬라이스를 함께 자름). uniform·aligned·cost-greedy(사이클 절감/중요도 손실이 큰 그룹부터, 비용 모델을 매 단계 호출) 세 전략.

### 3. 실험 설정

CIFAR-10 공식 학습 50,000장을 고정 시드로 학습 45,000 / 검증 5,000(클래스별 500)으로 나누고 검증 정확도로 체크포인트를 고르며 test 10,000장은 마지막에 한 번 평가한다. 모델 5개: ResNet-20 ReLU, ResNet-20 SiLU, MobileNetV2-0.5 ReLU6, 고객 A · ViT-128/6, 고객 B · Inception-32. 가상 NPU 프리셋 4개(tiny-1tops, edge-10tops, pcie-80tops, edge-10tops-strict). 캘리브레이션은 학습 분할에서 512장. 모든 학습·실험은 CPU 4코어에서 seed 0 한 번으로, 0.2%p 이하의 정확도 차이는 잡음이다(10k 표준오차  $\approx 0.3\%$ p).

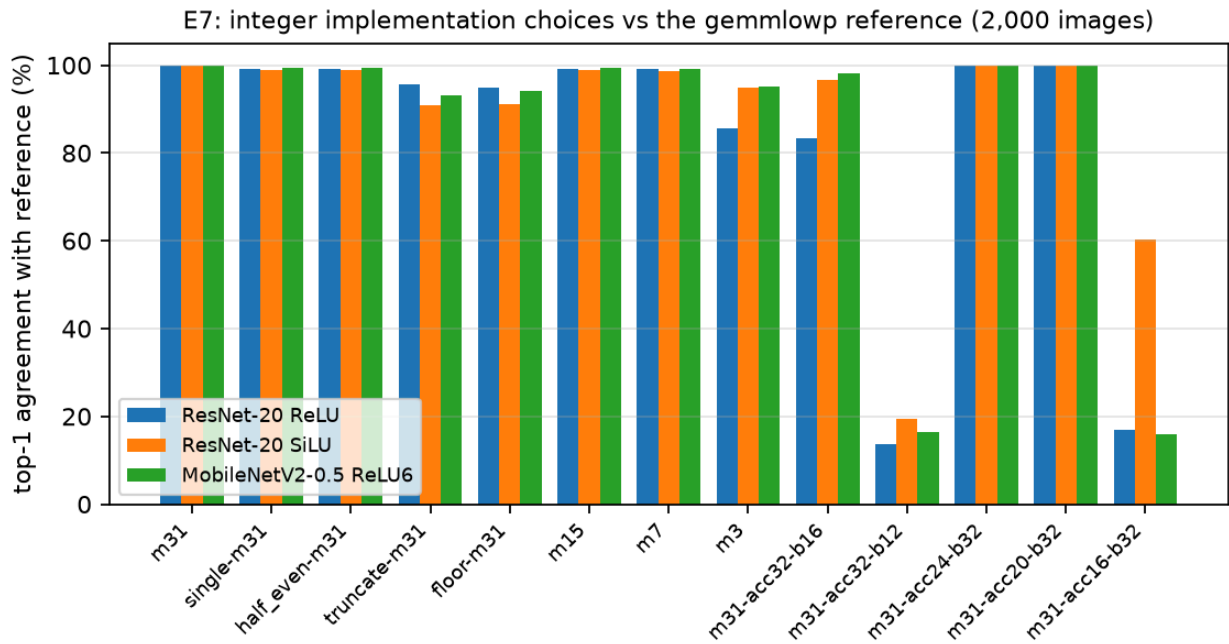
### 4. 결과

#### 4.1 fake-quant는 정확도를 맞히지만 텐서는 맞히지 않는다 (E2)

| 모델                    | 스킵          | 이미지    | fake-quant | 정수 엔진  | 차이      | top-1 일치 | 출력 코드 불일치 |
|-----------------------|-------------|--------|------------|--------|---------|----------|-----------|
| ResNet-20 ReLU        | npu-default | 10,000 | 89.54%     | 89.55% | +0.01%p | 98.8%    | 50%       |
| ResNet-20 ReLU        | per-tensor  | 10,000 | 89.61%     | 89.67% | +0.06%p | 99.6%    | 51%       |
| ResNet-20 SiLU        | npu-default | 10,000 | 90.35%     | 90.26% | -0.09%p | 98.4%    | 60%       |
| ResNet-20 SiLU        | per-tensor  | 10,000 | 90.17%     | 90.26% | +0.09%p | 99.2%    | 63%       |
| 고객 A · ViT-128/6      | npu-default | 10,000 | 80.87%     | 80.97% | +0.10%p | 98.0%    | 71%       |
| 고객 A · ViT-128/6      | per-tensor  | 10,000 | 80.77%     | 80.97% | +0.20%p | 98.4%    | 72%       |
| 고객 B · Inception-32   | npu-default | 10,000 | 89.55%     | 89.56% | +0.01%p | 100.0%   | 27%       |
| 고객 B · Inception-32   | per-tensor  | 10,000 | 89.52%     | 89.47% | -0.05%p | 99.2%    | 31%       |
| MobileNetV2-0.5 ReLU6 | npu-default | 10,000 | 90.30%     | 90.29% | -0.01%p | 99.6%    | 50%       |
| MobileNetV2-0.5 ReLU6 | per-tensor  | 10,000 | 90.23%     | 90.27% | +0.04%p | 100.0%   | 52%       |

주 스킵에서 정확도 차이는 최대 0.20%p, 이미지 단위 top-1 일치는 95.2~100.0%다. 그러나 출력 코드는 17~84%가 다르다. 각 conv의 국소 불일치는 0.02~0.93%(±1 LSB)뿐이며 나머지는 나비효과다.

#### 4.2 정수 구현 세부의 비용 (E7)

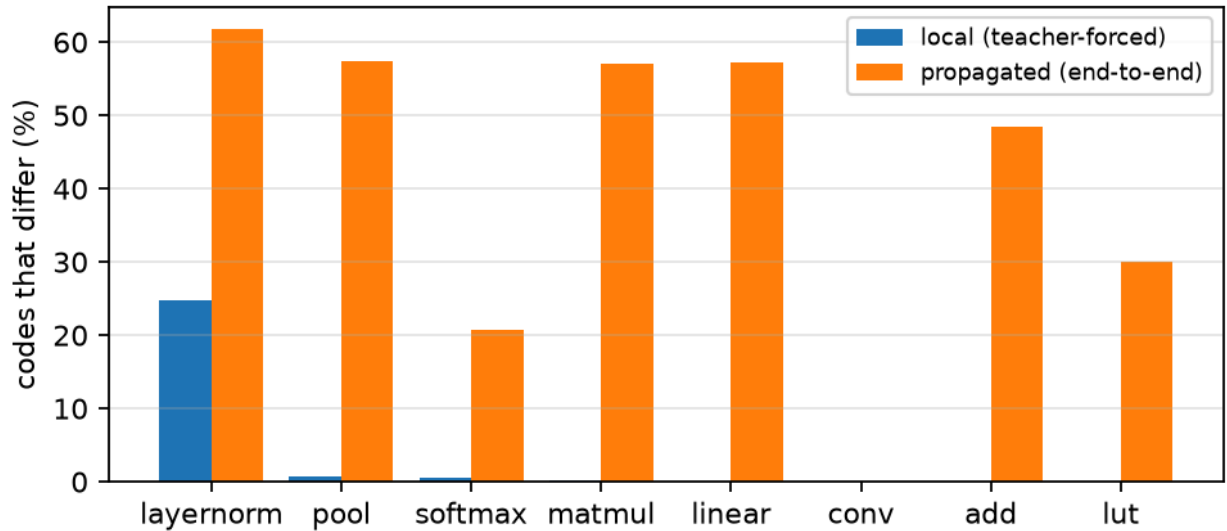


| RequantConfig (ResNet-20 ReLU) | 정확도    | 기준과 top-1 일치 | 포화 횟수      |
|--------------------------------|--------|--------------|------------|
| tfllite-m31-acc32-b32          | 90.10% | 100.0%       | 0          |
| single-m31-acc32-b32           | 90.10% | 99.2%        | 0          |
| half_even-m31-acc32-b32        | 90.20% | 99.2%        | 0          |
| truncate-m31-acc32-b32         | 89.45% | 95.6%        | 0          |
| floor-m31-acc32-b32            | 89.10% | 94.8%        | 0          |
| tfllite-m15-acc32-b32          | 90.20% | 99.2%        | 0          |
| tfllite-m7-acc32-b32           | 90.35% | 99.2%        | 0          |
| tfllite-m3-acc32-b32           | 81.90% | 85.7%        | 0          |
| tfllite-m31-acc32-b16          | 79.55% | 83.3%        | 0          |
| tfllite-m31-acc32-b12          | 13.45% | 13.6%        | 0          |
| tfllite-m31-acc24-b32          | 90.10% | 100.0%       | 0          |
| tfllite-m31-acc20-b32          | 90.10% | 100.0%       | 0          |
| tfllite-m31-acc16-b32          | 16.80% | 17.0%        | 11,152,196 |

반올림 모드(single/half-even)는 무관하지만 truncate/floor는 1~3%p를 잃고, 3비트 곱셈기는 ResNet-20 ReLU에서 -8%p, 바이어스 int16은 최대 -10%p, 누산기 int16은 모델을 무너뜨린다(1,115만 회 포화). 두 축은 해석에 한정이 붙는다. 바이어스 축은 int32 바이어스를 지수 조정 없이 잘라내는 구현의 비용이고(잘리는 코드 비율이 ReLU 4.7%, SiLU 1.0%, MobileNetV2 0.4%라 모델차가 크다; 시판 NPU는 더 넓다 — Vela는 Ethos-U 바이어스를 40비트로 패킹), 누산기 축은 K개를 다 더한 뒤 한 번 포화하는 최선 경우이며  $K \leq 1,280$  인 이 모델들에 한정된다(부분합 오버플로를 학습으로 보장하는 선행 연구: A2Q/A2Q+). 세 모델에서 등급 구분(무손실 · 손실 · 붕괴)은 같지만 손실 등급 안의 순서는 모델마다 다르다(ReLU: 바이어스 int16 > 3비트 곱셈기 > floor > truncate, SiLU: floor  $\approx$  truncate > 3비트 > 바이어스 int16, MobileNetV2: truncate > 3비트 > floor > 바이어스 int16).

#### 4.3 transformer에서 차이를 만드는 곳은 LayerNorm이다 (E9)

E9: fake-quant vs integer engine on the ViT, per op kind (64 images)

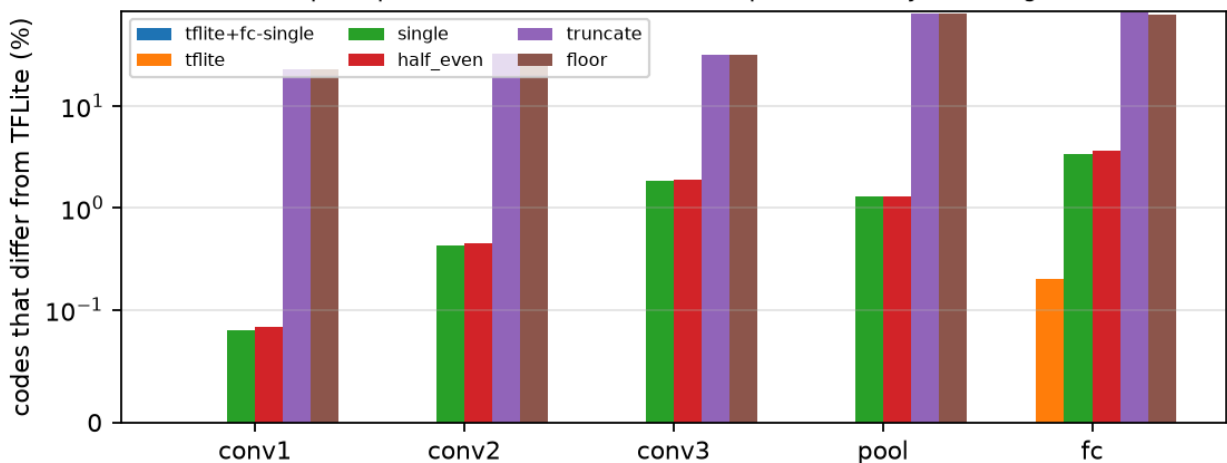


| op        | 노드 수 | 국소 불일치 평균 | 최대     | 전파    |
|-----------|------|-----------|--------|-------|
| layernorm | 13   | 24.74%    | 26.11% | 61.8% |
| pool      | 1    | 0.76%     | 0.76%  | 57.3% |
| softmax   | 6    | 0.49%     | 0.81%  | 20.8% |
| matmul    | 12   | 0.27%     | 0.43%  | 57.1% |
| linear    | 37   | 0.08%     | 0.16%  | 57.3% |
| conv      | 1    | 0.04%     | 0.04%  | 0.0%  |

ViT-128/6의 최종 출력 코드 불일치는 74%(CNN 30~50%)이고 top-1 일치는 96.9%다. 국소 불일치는 LayerNorm이 압도적이다: 정수 LayerNorm은 int64 합과 정확한 정수 제곱근으로 정규화하고 fake-quant는 float32로 계산한 뒤 격자에 올리므로 네 개 중 하나꼴로 반올림 경계 반대편에 떨어진다(모두  $\pm 1$  LSB).

#### 4.4 실제 런타임과의 비트 일치 (E11)

E11: npuloop vs TFLite reference kernels, per tensor, by rounding mode



| 반올림 conv.pool / fc    | 출력이 다른 이미지 | 다른 원소     | conv1 / conv2 / conv3 / pool / fc 국소 불일치 |
|-----------------------|------------|-----------|------------------------------------------|
| tflite / single       | 0/1000     | 0/10000   | 0.00% / 0.00% / 0.00% / 0.00% / 0.00%    |
| tflite / tflite       | 20/1000    | 20/10000  | 0.00% / 0.00% / 0.00% / 0.00% / 0.20%    |
| single / single       | 155/1000   | 337/10000 | 0.08% / 0.43% / 1.86% / 1.29% / 3.37%    |
| half_even / half_even | 182/1000   | 364/10000 | 0.09% / 0.44% / 1.91% / 1.30% / 3.64%    |

| 반올림 <b>conv·pool / fc</b> | 출력이 다른 이미지 | 다른 원소      | <b>conv1 / conv2 / conv3 / pool / fc</b> 국소 불일치 |
|---------------------------|------------|------------|-------------------------------------------------|
| truncate / truncate       | 1000/1000  | 8203/10000 | 23.10% / 32.78% / 32.13% / 80.23% / 82.03%      |
| floor / floor             | 1000/1000  | 7968/10000 | 23.10% / 32.78% / 32.13% / 80.23% / 79.68%      |

TFLite full-integer 모델의 파라미터를 그대로 읽어 만든 정수 그래프는 conv·pool을 gemmlowp 이중 반올림, fully-connected를 단일 반올림으로 실행할 때 TFLite reference 커널과 1,000장 × 모든 텐서에서 0개 불일치다. 그래프 전체를 한 모드로 두면 fc에서 0.2%가 ±1 LSB 어긋난다. TFLite 자신의 XNNPACK 델리게이트는 reference 커널과 155/1000장에서 다르다.

#### 4.5 실측과 모델을 같은 표에 (E10)

| 모델                    | NumPy ms/장 | C++ ms/장 | C++/NumPy | 두 엔진 출력 동일  | 모델 사이클 (edge-10tops) |
|-----------------------|------------|----------|-----------|-------------|----------------------|
| ResNet-20 ReLU        | 20.4       | 10.5     | 1.9×      | 2,000/2,000 | 34,417               |
| ResNet-20 SiLU        | 25.9       | 10.8     | 2.4×      | 2,000/2,000 | 34,785               |
| MobileNetV2-0.5 ReLU6 | 66.5       | 20.5     | 3.3×      | 2,000/2,000 | 87,056               |
| 고객 A · ViT-128/6      | 70.8       | 22.5     | 3.2×      | 2,000/2,000 | 52,246               |
| 고객 B · Inception-32   | 18.9       | 10.7     | 1.8×      | 2,000/2,000 | 31,224               |

이 저장소가 실측할 수 있는 시간은 호스트 CPU의 검증 엔진뿐이다. 처음 잔 C++ 엔진은 순진한 7중 루프라 NumPy(내부는 torch conv2d)보다 3배 느렸고, im2col + int32 GEMM(4탭 블록,  $K \cdot \max|x-zp| \cdot 128 < 2^{31}$  일 때만 int32 누산)으로 고쳐 2~3배 빠르게 만들었다.

#### 4.6 비용 모델 검증 (E8)

| 모델                    | 배열    | SCALE-Sim cycles | npuloop cycles | 비율     | 최악 레이어 오차 |
|-----------------------|-------|------------------|----------------|--------|-----------|
| ResNet-20 ReLU        | 32×32 | 84,276.0         | 84,298         | 1.0003 | 0.5%      |
| ResNet-20 ReLU        | 64×64 | 49,123.0         | 49,145         | 1.0004 | 0.5%      |
| ResNet-20 ReLU        | 16×16 | 208,486.0        | 208,508        | 1.0001 | 0.5%      |
| MobileNetV2-0.5 ReLU6 | 32×32 | 108,030.0        | 108,066        | 1.0003 | 0.2%      |
| MobileNetV2-0.5 ReLU6 | 64×64 | 70,074.0         | 70,110         | 1.0005 | 0.2%      |
| MobileNetV2-0.5 ReLU6 | 16×16 | 217,226.0        | 217,262        | 1.0002 | 0.1%      |
| 고객 B · Inception-32   | 32×32 | 96,108.0         | 96,124         | 1.0002 | 0.2%      |
| 고객 B · Inception-32   | 64×64 | 43,720.0         | 43,736         | 1.0004 | 0.2%      |
| 고객 B · Inception-32   | 16×16 | 289,122.0        | 289,138        | 1.0001 | 0.1%      |
| 고객 A · ViT-128/6      | 32×32 | 122,950.0        | 122,988        | 1.0003 | 0.3%      |
| 고객 A · ViT-128/6      | 64×64 | 49,620.0         | 49,658         | 1.0008 | 0.3%      |
| 고객 A · ViT-128/6      | 16×16 | 340,898.0        | 340,936        | 1.0001 | 0.3%      |

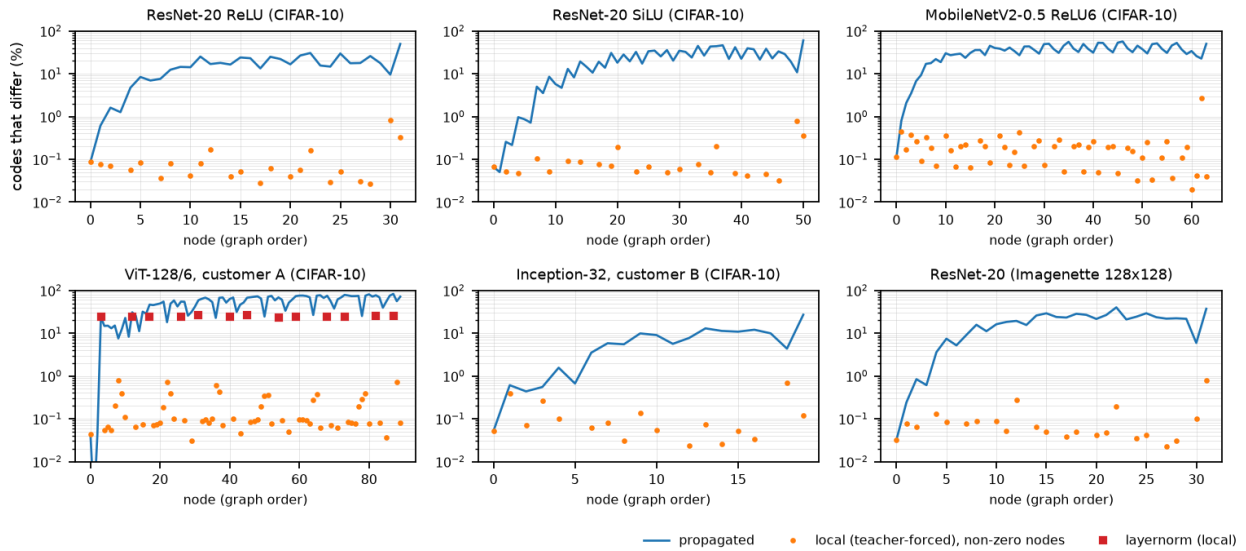
검증 범위는 dense conv·linear의 systolic 연산 사이클(단일 코어, 메모리 스톨 없음)이다. 처음 모델(M + R + C)은 10~13% 낙관적이었고 가중치 타일 적재 R 사이클이 빠져 있었다.

#### 4.7 두 번째 데이터셋 (E12)

Imagenette 128×128, ResNet-20(stem stride 2): val 84.70% → test 83.97%, 163M MACs; PTQ npu-default fake 84.08% / 정수 84.03%, per-tensor fake 83.97% / 정수 84.05%, pow2 fake 84.08% / 정수 83.95%

## 4.8 같은 이미지에서 켜 오차 막대 (E15)

E15: where the fake-quant / integer gap is born (local) and where it goes (propagated)



그림과 국소/전파 열은 배치 250장(Imagenette는 125장)에서, 표의 정확도·코드 열은 test 전체에서 켜 값이다.

| 모델                         | 데이터셋           | 스킵          | test   | fake-quant | 정수 엔진  | 정수 - fake (쌍 SE) | 정답 여부가 같은 장수 | top-1 일치 | 출력 코드 불일치 (전체) |
|----------------------------|----------------|-------------|--------|------------|--------|------------------|--------------|----------|----------------|
| ResNet-20 ReLU             | CIFAR-10       | npu-default | 10,000 | 89.54%     | 89.55% | +0.01 ± 0.08%p   | 57           | 99.37%   | 50.1%          |
| ResNet-20 ReLU             | CIFAR-10       | per-tensor  | 10,000 | 89.61%     | 89.67% | +0.06 ± 0.08%p   | 72           | 99.18%   | 51.7%          |
| ResNet-20 SiLU             | CIFAR-10       | npu-default | 10,000 | 90.35%     | 90.26% | -0.09 ± 0.09%p   | 79           | 99.06%   | 61.6%          |
| ResNet-20 SiLU             | CIFAR-10       | per-tensor  | 10,000 | 90.17%     | 90.26% | +0.09 ± 0.09%p   | 73           | 99.07%   | 62.9%          |
| MobileNetV2-0.5 ReLU6      | CIFAR-10       | npu-default | 10,000 | 90.30%     | 90.29% | -0.01 ± 0.08%p   | 61           | 99.28%   | 50.4%          |
| MobileNetV2-0.5 ReLU6      | CIFAR-10       | per-tensor  | 10,000 | 90.23%     | 90.27% | +0.04 ± 0.07%p   | 54           | 99.37%   | 51.4%          |
| 고객 A · ViT-128/6           | CIFAR-10       | npu-default | 10,000 | 80.87%     | 80.97% | +0.10 ± 0.13%p   | 160          | 97.93%   | 72.7%          |
| 고객 A · ViT-128/6           | CIFAR-10       | per-tensor  | 10,000 | 80.77%     | 80.97% | +0.20 ± 0.12%p   | 152          | 98.07%   | 72.6%          |
| 고객 B · Inception-32        | CIFAR-10       | npu-default | 10,000 | 89.55%     | 89.56% | +0.01 ± 0.05%p   | 27           | 99.69%   | 28.5%          |
| 고객 B · Inception-32        | CIFAR-10       | per-tensor  | 10,000 | 89.52%     | 89.47% | -0.05 ± 0.06%p   | 39           | 99.52%   | 31.1%          |
| ResNet-20 (Imagenette-128) | Imagenette-128 | npu-default | 3,925  | 84.05%     | 83.97% | -0.08 ± 0.10%p   | 15           | 99.52%   | 35.7%          |
| ResNet-20 (Imagenette-128) | Imagenette-128 | per-tensor  | 3,925  | 84.00%     | 84.05% | +0.05 ± 0.11%p   | 20           | 99.31%   | 36.2%          |

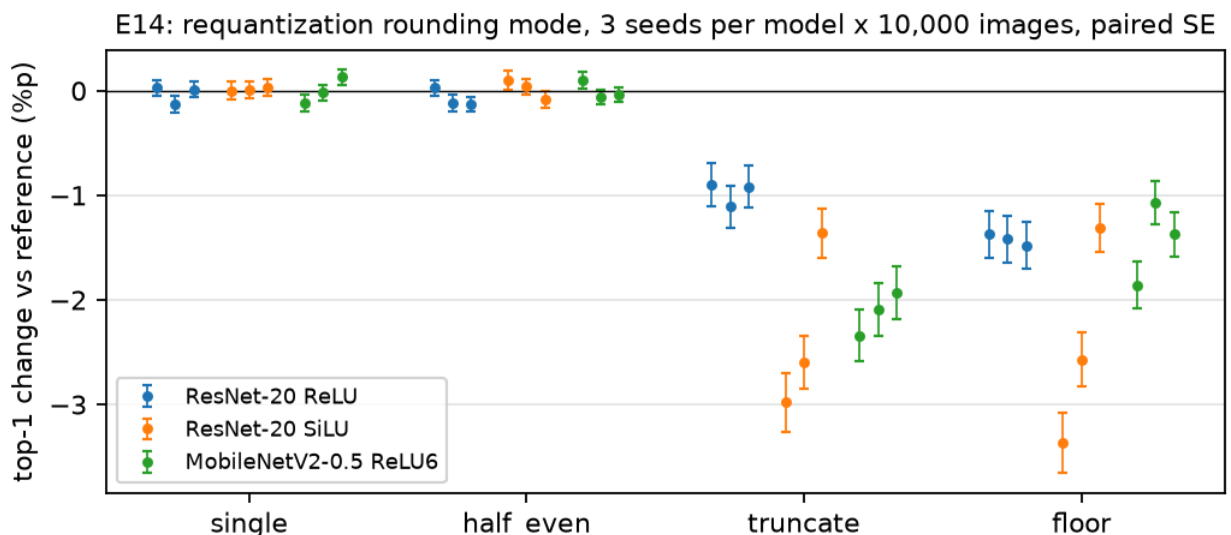
4.1의 차이에는 표준오차가 없었다. 여기서는 test 전체를 같은 이미지에서 세 번(FP32·fake-quant·정수) 평가 해 (정수 정답 - fake 정답)의 표본 표준편차로 쌍 표준오차를 구했다. 이 표에서 가장 큰 정확도 차이는 0.20%p, 가장 큰  $|\Delta|/SE$ 는 1.6로 12행 모두 2 미만이다(4.9의 LayerNorm 에뮬레이션 표까지 합치면 최대 2.2). E15·E16 16개 비교를 다중비교 보정하지 않은 값이므로, 이 표본 크기에서 fake-quant와 정수 정확도의 차이는 잡음과 구분되지 않는다. 출력 코드는 test 전체에서 28~73%가 다르다; Imagenette 128×128의 ResNet-20도 같은 그림이다(코드 36%, top-1 일치 99.5%). 이 표는 모든 모델을 512장으로 통일해 다시 캘리브레이션했으므로 4.7(E12, Imagenette 256장)의 정확도와 소수점 이하가 다르다.

#### 4.9 LayerNorm을 정수로 에뮬레이션하면 (E16)

| 스킵          | fake-quant의 LayerNorm | fake-quant | 정수 엔진  | 정수 - fake (쌍 SE) | top-1 일치 | 출력 코드 불일치 | LN 국소  | softmax 국소 | matmul 국소 |
|-------------|-----------------------|------------|--------|------------------|----------|-----------|--------|------------|-----------|
| npu-default | float32               | 80.87%     | 80.97% | +0.10 ± 0.13%p   | 97.93%   | 72.7%     | 24.77% | 0.50%      | 0.27%     |
| npu-default | 정수 에뮬레이션 (13개)        | 80.72%     | 80.97% | +0.25 ± 0.11%p   | 98.29%   | 65.7%     | 0.00%  | 0.50%      | 0.27%     |
| per-tensor  | float32               | 80.77%     | 80.97% | +0.20 ± 0.12%p   | 98.07%   | 72.6%     | 24.76% | 0.50%      | 0.27%     |
| per-tensor  | 정수 에뮬레이션 (13개)        | 80.84%     | 80.97% | +0.13 ± 0.10%p   | 98.60%   | 66.0%     | 0.00%  | 0.50%      | 0.27%     |

4.3의 진단이 맞다면 fake-quant의 LayerNorm을 정수 엔진의 산술로 바꾸는 것만으로 transformer의 격차가 줄어야 한다. npuloop.quant.emulate 는 캘리브레이션된 그래프의 LayerNorm을 정수 엔진과 같은 함수로 계산하는 모듈로 바꾼다(export는 그대로, 엔진 쪽 정수 프로그램은 동일). ViT-128/6(npu-default)에서 LayerNorm 국소 불일치는 24.77% → 0.00%, 출력 코드 불일치는 72.7% → 65.7%, top-1 일치는 97.93% → 98.29%, 정확도 차이는 +0.10 ± 0.13 → +0.25 ± 0.11%p가 된다. 남은 격차는 softmax·matmul·linear의 국소 ±1 LSB와 그 전파다. 정확도·쌍 SE·top-1 일치·출력 코드 열은 test 10,000장 전체에서, 국소 불일치 열은 250장 배치의 teacher-forced 비교에서 나온 값이다.

#### 4.10 반올림 모드, 시드 셋 (E14)



| 2단계 반올림 | ResNet-20 ReLU: 기준 대비 $\Delta acc$ , 시드 평균 ± 표준편차 (시드별) | ResNet-20 SiLU: 기준 대비 $\Delta acc$ , 시드 평균 ± 표준편차 (시드별) | MobileNetV2-0.5 ReLU6: 기준 대비 $\Delta acc$ , 시드 평균 ± 표준편차 (시드별) |
|---------|---------------------------------------------------------|---------------------------------------------------------|----------------------------------------------------------------|
| single  | -0.03 ± 0.09 (+0.03, -0.13, +0.01)                      | +0.01 ± 0.02 (+0.00, +0.01, +0.03)                      | -0.00 ± 0.13 (-0.12, -0.02, +0.13)                             |

| 2단계 반올림   | ResNet-20 ReLU: 기준 대비 $\Delta acc$ , 시드 평균 $\pm$ 표준편차 (시드별) | ResNet-20 SiLU: 기준 대비 $\Delta acc$ , 시드 평균 $\pm$ 표준편차 (시드별) | MobileNetV2-0.5 ReLU6: 기준 대비 $\Delta acc$ , 시드 평균 $\pm$ 표준편차 (시드별) |
|-----------|-------------------------------------------------------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| half_even | $-0.07 \pm 0.09$ (+0.03, -0.12, -0.13)                      | $+0.02 \pm 0.09$ (+0.10, +0.04, -0.08)                      | $+0.00 \pm 0.09$ (+0.10, -0.06, -0.04)                             |
| truncate  | $-0.98 \pm 0.12$ (-0.90, -1.11, -0.92)                      | $-2.31 \pm 0.85$ (-2.98, -2.60, -1.36)                      | $-2.12 \pm 0.21$ (-2.34, -2.09, -1.93)                             |
| floor     | $-1.42 \pm 0.06$ (-1.37, -1.42, -1.48)                      | $-2.42 \pm 1.04$ (-3.37, -2.57, -1.31)                      | $-1.43 \pm 0.40$ (-1.86, -1.07, -1.37)                             |

4.2의 반올림 축을 시드 3개  $\times$  test 10,000장에서 다시 잰다(기준: gemmlowp 이중 반올림, 같은 이미지의 쌍 SE). 강건한 것은 부호와 유의성이다: truncate-floor는 측정된 18개 조합 모두에서 기준보다 나쁘고  $|\Delta|/SE$ 가 4.4~11.8다. 크기는 그렇지 않다 — 시드 표준편차가 가장 작은 ResNet-20 ReLU의 floor는  $-1.42 \pm 0.06\%$ p 인 반면 가장 큰 ResNet-20 SiLU의 floor는  $-2.42 \pm 1.04\%$ p(-3.37~-1.31)로, 시드 표준편차가 효과의 43%에 이른다. 두 모드의 순서도 고정되지 않는다: 9개 (모델, 시드) 중 floor가 더 나쁜 경우는 4개다. 5장이 E7에 대해 적은 경고("체크포인트를 바꾸면 크기가 달라진다")가 반올림 축에서도 성립하므로, 한 시드의 손실값을 그 모드의 비용으로 인용해서는 안 된다.

## 5. 한계

CIFAR-10 규모, seed 1개(E14의 반올림 축만 3개), 가상 NPU(실제 컴파일러의 fusion·타일링·메모리 스케줄링 없음), 에너지는 자릿수 추정, TFLite 교차 검증은 conv·pool·fc에 한정, ImageNet 사전학습 체크포인트 미사용(호스트 차단). 체크포인트를 바꾸면 결론 일부가 흔들렸다: E3의 레이어 수준 상관은 재현되지 않았고 E7의 3비트 곱셈기 손실은 체크포인트에 따라 -1.3에서 -8.2%p까지 달랐다.

## 6. 결론

fake-quant 정확도는 믿어도 되지만 텐서는 믿으면 안 되고, "INT8"은 런타임마다 다른 반올림을 뜻하며, 프루닝의 값은 사이클로 환산했을 때만 드러난다. 이 세 문장을 재현 가능한 숫자로 만든 것이 npuloop이다.

## 참고문헌

Jacob et al. 2018 (CVPR) 정수 산술 추론; gemmlowp/TFLite `MultiplyByQuantizedMultiplier`; Samajdar et al. 2020 SCALE-Sim; Nagel et al. 2019 DFQ; Liu et al. 2017 Network Slimming; Li et al. 2021 MQBench; Kim et al. 2021 I-BERT; Horowitz 2014 (ISSCC) 에너지 수치.